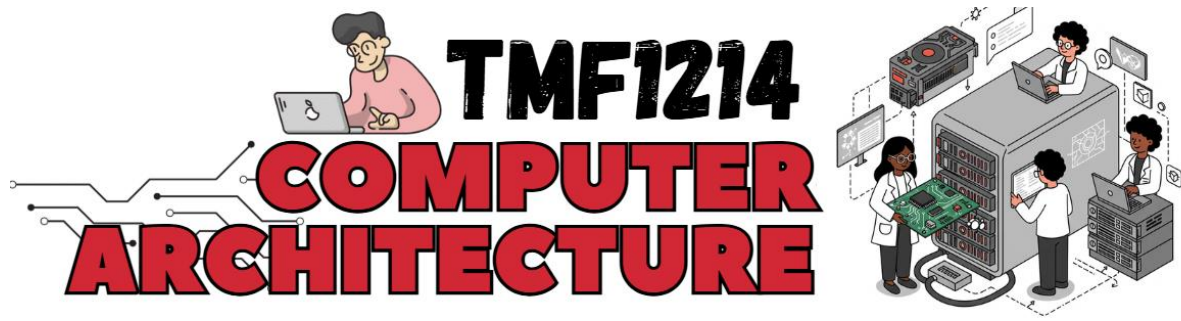




GROUP PROJECT

BUILDING A WEB-BASED ISA SIMULATOR (20%)

General Information

Semester : Sem 2 Sesi 2025/2026
Type : Group (max 8 per group)
Weightage : 20% of final marks
Released Date : 13th April 2026 (Monday)(Week 7)
Submission Date : 5th June 2026 (Friday)(Week 13), by 11.59pm through eLEAP

Instructions

1. Form a group consisting of eight (8) students and register your group members using the link provided on the eLeap page.
2. Students are required to answer **ALL** questions and complete the tasks outlined in this document.
3. Ensure all group members contribute actively. Be ready to answer questions about your work.
4. This project is worth 20% of the continuous assessment

1. INTRODUCTION

In this project, you will bridge the gap between high-level programming and low-level hardware execution. You are provided with a "Starter Kit" consisting of a virtual CPU environment. Your task is to extend this environment to handle advanced arithmetic and control flow logic.

2. SETUP INSTRUCTIONS

You are provided with two core files: index.html and simulator.js.

1. Place both files in the **same folder**.
2. **Run the Simulation:** Double-click index.html to open it in your web browser.
3. **Initial Test:** Ensure the pre-loaded code runs correctly:

Code snippet

```
LOAD R1 10
LOAD R2 20
ADD R3 R1 R2
```

4. Click **"Execute Program"**. Register **R3** should display **30**.

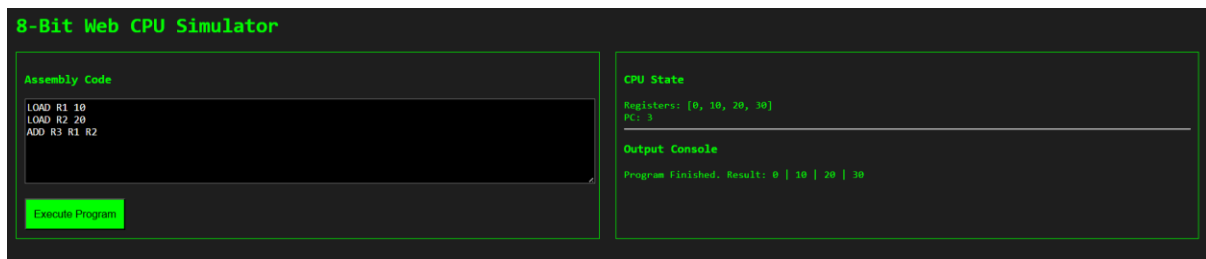


Fig. 1 : Example interface for simulator

3. TASK: EXTENDING THE CPU

You must modify simulator.js (and index.html if necessary for UI improvements) to support the following architectural features:

A. Advanced Arithmetic (The Math Unit)

Modify the logic to support the following instructions:

- **MUL (Multiplication):** MUL R1 R2 R3 (Result: $R1 = R2 * R3$)
- **DIV (Division):** DIV R1 R2 R3 (Result: $R1 = R2 / R3$)
- **Fraction Support:** Modify the register handling so the CPU can store and process decimal numbers (e.g., LOAD R1 10.5).

B. Control Flow (The Branch Unit)

Implement "Smart" instructions that allow the program to loop or jump:

- **JUMP:** JUMP [LineNumber] — Forces the Program Counter (PC) to move to a specific line.
- **BEQ (Branch if Equal):** BEQ R1 R2 [LineNumber] — If the values in R1 and R2 are equal, jump to the specified line.

C. Hardware Exception Handling

A robust architecture must handle errors gracefully. Implement code to catch and report:

- **Division by Zero:** The simulation should stop and display an error message.
- **Invalid Register Access:** If a user calls R99 or a non-existent register, the system must report an "Invalid Register" error.

4. VALIDATION (THE "CALCULATOR" CHALLENGE)

To prove your simulator works, you must write a short Assembly program in your tool that calculates the following:

Challenge: Write an Assembly script that calculates $15.5 / 2$ then adds the result to 10 and finally uses a **BEQ** loop to multiply the final result by 3.

What the Final Script Looks Like

If the simulator is working correctly, the final test script should look similar to this:

```
; Phase 1 & 2: Calculate (15.5 / 2) + 10
LOAD R1 15.5
LOAD R2 2
DIV R3 R1 R2    ; R3 = 7.75
LOAD R4 10
ADD R5 R3 R4    ; R5 = 17.75 (This is our "Base")

; Phase 3: Multiply R5 by 3 using a loop
LOAD R0 0       ; R0 will be our Final Result (Accumulator)
LOAD R6 3       ; R6 is our Loop Counter
LOAD R7 1       ; Constant for decrementing

; --- Loop Start (Line 8) ---
ADD R0 R0 R5    ; Add 17.75 to the total
SUB R6 R6 R7    ; Decrement the counter
BEQ R6 R10 12   ; If counter is 0 (R10 is empty), jump to HALT
JUMP 8          ; Else, go back to Line 8

HALT           ; Final R0 should be 53.25
```

Fig.2 : Expected script for Validation Challenge

5. DELIVERABLES

Each group must submit:

1. **Source Files:** The modified index.html and simulator.js.
2. **Demo Video:** A short (2-5minute) screen recording showing your simulator running the "Calculator Challenge" **OR** a group presentation arranged by your lecturer (*whichever option preferred by the lecturer)
3. **Report:** Explanation of how you implemented the tasks

6. SUBMISSION DETAILS

- **Deadline:** 5th June 2026 (Friday)
- **Submission Platform:** eLEAP
- **File Naming Format:** TMF1214-CA-ProjectReport-GroupNo.pdf (with video links) plus simulator.js and index.html
- No hardcopy submission needed
- **Note:** Late submissions will receive **zero (0) marks**.

Grading Rubric

Criteria	Weight	Excellent (20–15)	Good (14–10)	Fair (9–1)	Poor (0)
Instruction Implementation	20%	Flawless execution of MUL, DIV, JUMP, and BEQ opcodes.	Majority of instructions are functional with minor logic errors.	Only 1 or 2 instructions work correctly; partial logic.	Instructions are not implemented or do not function.
Fraction/Float Support	20%	Full support for non-integers; parseFloat works perfectly across all math.	Most operations handle decimals, but some precision is lost.	Minimal support; registers struggle with non-integers.	Only handles whole integers; fails on decimal input.
Error Handling	20%	Robust "Trap" logic for Division-by-Zero and Invalid Registers (e.g., R99).	Basic error messages appear, but program might not halt safely.	Inconsistent error catching; some invalid inputs cause crashes.	No error handling; simulator crashes on invalid inputs.
The Challenge Script	20%	Fully functional script that completes the calculation and looping challenge.	Script runs but requires minor manual intervention or has a logic bug.	Partial script execution; fails to complete the loop or math.	No assembly script provided or script does not run.
Report & Presentation	20%	Documentation is professional, clear, and presentation is excellent.	Report is clear, but lacks technical depth in architectural explanation.	Report is brief/unorganized; presentation is average.	Poor quality documentation and presentation.
Total	100				

Final Grade Calculation:

Final mark = Total Score/100* 20%